



区块链数字文档存证系统编程实践任务书

AI + 区块链融合应用

密码学核心应用编程课程实践

一、基本信息



课程名称

密码学核心应用编程



实践主题

区块链数字文档存证系统开发



适用对象

计算机专业大三学生



实践时长

4课时（每课时45分钟）



前置知识

Python基础、密码学基本概念



提交期限

实践结束后一周内

二、学习目标



哈希算法应用

掌握SHA-256哈希算法在文档完整性验证中的应用。



默克尔树原理

理解默克尔树数据结构及在区块链中的工作原理。



RSA非对称加密

熟练使用RSA非对称加密算法实现安全数据传输。



数字签名技术

掌握数字签名技术确保操作的不可否认性。



构建完整存证系统

综合应用三大密码学原语构建完整存证系统。



区块链与AI融合

体验区块链与AI技术融合的实际工作流程。

三、任务描述



文档完整性保障

通过哈希计算和默克尔树结构确保文档内容不可篡改，构建坚实的数据基础。



安全数据传输

使用非对称加密技术，对文档摘要进行加密处理，保障传输过程中的数据机密性。



操作真实性验证

引入数字签名机制，确保每一次存证操作的来源合法且真实，实现操作的不可否认性。



AI内容存证

针对AI生成的文档内容，设计并实现完整的区块链化存证流程，确保生成内容的权属清晰。

四、技术要求



编程语言环境

核心版本：Python 3.8+

确保支持现代语法特性及加密库兼容性



核心依赖库

cryptography (RSA加密/数字签名)

安装命令：pip install cryptography



内置辅助库

hashlib, json, base64, time

用于处理哈希计算、数据序列化及编码



开发工具 IDE

PyCharm 或 VS Code

推荐使用带有代码补全和调试功能的现代化编辑器

五、详细任务分解（第1课时）



核心任务：文档创建与完整性验证



文档准备

创建4个模拟文档



哈希计算函数实现

编写 `compute_file_hash()`，计算文件的SHA-256值



默克尔树构建

构建两层默克尔树结构，计算并输出根哈希



完整性验证测试

实现验证逻辑，测试文档被篡改后的结果



交付成果与验收标准



哈希值输出

控制台清晰展示4个文档的完整SHA-256哈希值



默克尔根验证

程序输出正确的默克尔根哈希，确保计算逻辑无误



篡改检测演示

手动修改文档内容后，验证函数能准确返回失败结果

五、详细任务分解（第2课时）

核心任务：安全传输模块实现

密钥生成

生成 RSA-2048 位密钥对（公钥/私钥）

加密函数实现

实现 `encrypt_digest`，使用 OAEP 填充加密哈希值

解密函数实现

实现 `decrypt_digest`，解密并还原原始哈希值

全流程测试

测试完整的加密-传输-解密闭环流程

交付与验收标准

密钥对生成成功

程序运行后能正确输出公私钥文件或对象

流程演示完整

加密传输解密后的哈希值与原始值完全一致

安全性验证通过

使用错误的私钥进行解密操作时，系统应明确抛出异常或返回失败

五、详细任务分解（第3课时）

核心任务：数字签名与存证操作

- ✓ 设计存证记录数据结构，定义关键字段
- ✓ 实现签名生成函数与验签函数
- ✓ 编写单元测试，覆盖三种核心验证场景

存证记录数据结构定义

-  基础信息：文档ID、文件哈希(Hash)、时间戳
-  元数据：所有者信息、操作类型
-  来源标识：内容来源（如 AI_GENERATED）

核心验证场景测试

- ✓ 场景一：正常记录验证（预期结果：通过）
- ✗ 场景二：篡改时间戳后验证（预期结果：失败）
- ✗ 场景三：使用错误公钥验证（预期结果：失败）

任务交付标准

- ☐ 代码实现完整的签名生成与验证流程闭环
- ☐ 三个测试场景结果均符合预期，无异常报错
- ☐ 最终签名结果必须以 Base64 格式进行输出

五、详细任务分解（第4课时）

核心任务：系统集成与优化



模块整合与流程编排

整合前三个模块，确保程序按逻辑顺序执行



输出格式优化

增加步骤说明，使日志输出更清晰美观



个性化身份验证

添加个性化学号哈希，实现唯一身份绑定



健壮性完善

完善异常处理机制，补充详细代码注释



标准报告输出

生成包含系统状态的标准总结报告

最终输出格式示例 (CLI Report)



===== 区块链文档存证系统模拟完成 =====

文档完整性模块: [✓] 完成

- AI生成文档: patent_2024_001.txt

- 文档数量: 4 | 默克尔根哈希: 0xabc123...

安全传输模块: [✓] 完成

- 加密算法: AES-256-GCM | 签名算法: ECDSA

身份验证模块: [✓] 完成

- 学号哈希验证通过 | 角色: Admin

总体验证: [✓] 所有密码学操作验证成功

系统状态: 可投入模拟运行

六、评分标准（总分100分）

功能完整性 (65分)

- AI内容存证 (10) / 系统集成 (10)
- 哈希与默克尔树 (15) / 非对称加密 (15)
- 数字签名 (15)

代码质量 (30分)

- 代码结构清晰 (10) / 注释与文档完善 (10)
- 异常处理机制 (5)
- 符合编码规范 (5)

创新与扩展 (10分)

- 扩展功能实现 (5)
- 输出界面美观性 (5)

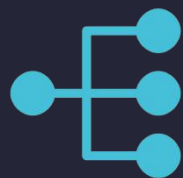
防抄袭验证 (5分)

- 学号哈希输出验证 (5)

扣分项

- 代码无法运行 (-20)
- 核心功能缺失 (每项-10)

七、扩展挑战（加分项）



动态默克尔树

+5 分

支持任意数量文档的默克尔树构建，实现通用函数，提升系统的灵活性。



真实文件支持

+3 分

支持读取图片、PDF等真实文件，实现大文件的分块哈希计算功能。



智能错误提示

+2 分

为验证失败场景提供精准定位信息，如指出具体被篡改的文档编号。

八、开发支持与资源



核心库安装

执行以下命令安装实验所需的加密库：

```
pip install cryptography
```



常见问题解答

- 中文编码处理方案
- 哈希值拼接与转换
- 密钥安全保存策略
- 填充模式区别与选择



时间管理建议

建议按4个课时分配工作：

- 课时1-2：环境搭建与基础编码
- 课时3：核心功能实现与测试
- 课时4：优化与文档整理

九、提交要求



提交文件清单

- 必须包含：源代码文件 (.py / .java 等)、程序运行截图
- 可选提交：详细的设计说明文档



代码头部规范

- 强制要求：必须包含作者姓名、学号、班级信息
- 补充信息：需注明程序完成日期及主要功能简述



运行与输出要求

- 可执行性：代码必须能够一键运行，无环境依赖报错
- 输出验证：程序末尾需输出个性化学号哈希值



打包与提交方式

- 文件打包：将所有相关文件压缩为 ZIP 格式
- 提交渠道：通过指定的课程平台作业提交入口上传

十、学术诚信声明



独立完成原则

本实践为个人独立完成项目，鼓励思想交流与讨论，但严禁直接复制他人代码或解决方案。



代码身份验证

提交的代码必须包含个性化学号哈希，系统将据此进行身份验证，确保提交内容归属正确。



雷同零容忍

如发现代码高度雷同，所有相关方均按0分处理，情节严重者将上报学院进行纪律处分。



规范引用来源

允许参考官方文档和开源代码，但必须在代码注释中明确注明来源链接或作者，尊重知识产权。

实践开始前确认



环境准备：Python 3.8+

确认开发环境已搭建完毕，版本符合要求，确保代码能正常运行。



依赖安装：Cryptography库

已执行pip install命令安装所需加密库，避免运行时出现导入错误。



任务认知：理解实践要求

已通读任务书，明确本次实践的目标、步骤及预期交付成果。



时间规划：4课时连续时间

确保拥有连续且不受打扰的4个课时时间，以保证实践的连贯性。

祝您实践顺利，学有所获！

注：本任务书最终解释权归课程组所有。如有问题，请及时与指导老师沟通。